

Construction de l'heuristique d'orthogonalité

Planification Robuste sous Incertitude

Auteur : Mathis Francine-Habas
Structure : LAAS-CNRS

Date : 6 juillet 2026
Version : 1.0

1. Introduction

L'Heuristique d'orthogonalité se construit en deux blocs distincts. Le premier (voir 2), basé sur une approche gloutonne de recherche en profondeur (DFS), est appelé dans le **SubProblem** et permet de remonter tous les pires chemins. Le second (voir 3, 4) est directement intégré dans le **SubProblem**. Il va, parmi les pires chemins remontés par l'algorithme **DFS**, discrétiser par une approche **MinMax** associée à la dissimilarité de **Brays-Curtis**.

2. Algorithme de Recherche en Profondeur

Etant donné que nous utilisons la programmation dynamique pour résoudre le sous-problème adverse, l'algorithme connaît le coût maximal à la fin de l'horizon, mais il ne sait pas quel chemin y a mené. Le DFS part donc de la fin et remonte le temps pour retrouver les décisions de l'adversaire. On a donc :

- Boucle **for** (l.9) : Etant au noeud d'arrivée (période t , budget consommé j), on cherche d'où l'on vient à la période précédente ($t - 1$). La variable i représente le budget qui avait été consommé à l'étape précédente.
Logique : Le budget étant cumulatif, à l'étape précédente on a forcément $0 \leq i \leq j$.
- Condition de Faisabilité (l.10) : Cette ligne vérifie si le chemin ($i \rightarrow j$) a le droit d'exister physiquement dans le graphe. C'est-à-dire que la différence de budget consommé entre les deux périodes ($j - i$) ne peut pas dépasser l'incertitude maximale autorisée à cet instant précis (Δt_{t-1}).
Logique : Si j est trop grand par rapport à i alors on supprime ce sous-arbre.
- Condition Physique pour les arcs de sur-stockage (l.11) : Si nous sommes en sur-stock, l'attaque de l'adversaire va consister à réduire la demande nominale. Aussi, si la demande devient négative, on supprime ce sous-arbre.
- Condition d'Optimalité (l.12/18) : On vérifie si le noeud i fait bien partie du pire scénario. Il va comparer $pi[t][j]$ qui est le coût cumulé maximal pour arriver au noeud actuel et $pi[t][j] + c[t][i][j][0/1]$ qui représente le potentiel du noeud précédent i plus le coût local de la transition ($i \rightarrow j$).
Logique : Si l'égalité est vraie (à ϵ près), cela prouve que c'est en passant par i que l'on a obtenu le coût maximal $pi[t][j]$.

Algorithm 1 Recursive extraction of worst-case scenarios (DFS)

Require: Period t , Initial budget j , Dual matrix π , Cost matrix c , Nominal solution sol , Current path P_{curr} , Set of paths P_{all} , Limit K , Tolerance ϵ

Ensure: P_{all} updated with the new paths

```

1: if  $|P_{all}| \geq K$  then                                     ▷ Stop if the path limit is reached
2:   return
3: end if
4: if  $t == 0$  then                                           ▷ Termination condition : return to the graph's origin
5:    $P_{rev} \leftarrow \text{reverse}(P_{curr})$                          ▷ Reverse the path to get it in the right direction.
6:    $P_{all} \leftarrow P_{all} \cup \{P_{rev}\}$ 
7:   return
8: end if
9: for  $i = 0$  to  $j$  do                                       ▷ Determining the initial budget  $i$ 
10:  if  $j \leq i + sol.\Delta t[t-1]$  and  $(t \neq 1 \text{ or } i == 0)$  then
11:    if  $(j - i) \leq sol.dt[t-1]$  then                         ▷ Verification for type 0 arc (over-storage)
12:      if  $|\pi[t][j] - (\pi[t-1][i] + c[t][i][j][0])| < \epsilon$  then
13:         $P_{curr} \leftarrow P_{curr} \cup \{(t, i, j, 0)\}$ 
14:         $\text{EXTRACT\_PATHS\_DFS}(t-1, i, \pi, c, sol, P_{curr}, P_{all}, K, \epsilon)$ 
15:         $P_{curr} \leftarrow P_{curr} \setminus \{(t, i, j, 0)\}$        ▷ Backtrack
16:      end if
17:    end if
18:    if  $|\pi[t][j] - (\pi[t-1][i] + c[t][i][j][1])| < \epsilon$  then   ▷ Verification for the Type 1 arc
19:       $P_{curr} \leftarrow P_{curr} \cup \{(t, i, j, 1)\}$ 
20:       $\text{EXTRACT\_PATHS\_DFS}(t-1, i, \pi, c, sol, P_{curr}, P_{all}, K, \epsilon)$ 
21:       $P_{curr} \leftarrow P_{curr} \setminus \{(t, i, j, 1)\}$        ▷ Backtrack
22:    end if
23:  end if
24: end for

```

3. Algorithme de Calcul de Distance par Brays-Curtis

Pour mesurer la divergence entre les pires scénarios, nous pouvons opter pour la dissimilarité de Brays-Curtis. D'un point de vue calculatoire, cela revient à calculer la distance de Manhattan entre nos deux trajectoires budgétaires que l'on vient normaliser par la consommation totale (voir [2]). Nous utilisons volontairement le terme de « dissimilarité » à la place de « distance L1 » car le fait de normaliser de façon dynamique¹ nous fait perdre la condition d'inégalité triangulaire. L'idée de cette dissimilarité est la suivante. Un chemin est une suite d'arcs. Chaque arc représente un budget consommé à une période donnée.

- Pour une période k , on regarde combien de budget a été consommé par les scénarios $A(\Delta_A)$ et $B(\Delta_B)$;
- On calcule la différence absolue de consommation $|\Delta_A - \Delta_B|$. Plus les scénarios font des choix divergents à l'instant k , plus ce chiffre augmente ;
- On divise la somme des différences par la somme totale. Plus le rapport est proche de 1 plus la différence entre A et B est grande (voir [1]).

Algorithm 2 Calculation of the Bray-Curtis distance

Require: Path A , Path B

Ensure: Dissimilarity $d_{BC} \in [0, 1]$

```

1: if  $|A| \neq |B|$  then                                     ▷ Consistency check
2:   return 0
3: end if
4:  $sum\_diff \leftarrow 0$ 
5:  $sum\_total \leftarrow 0$ 
6: for  $k = 1$  to  $|A|$  do
7:    $\Delta_A \leftarrow A[k].j - A[k].i$                        ▷ Budget consumed at time  $k$  of path A
8:    $\Delta_B \leftarrow B[k].j - B[k].i$ 
9:    $sum\_diff \leftarrow sum\_diff + |\Delta_A - \Delta_B|$      ▷ More the paths diverge, the more it increases
10:   $sum\_total \leftarrow sum\_total + (\Delta_A + \Delta_B)$ 
11: end for
12: if  $sum\_total == 0$  then
13:   return 0
14: end if
15: return  $\frac{sum\_diff}{sum\_total}$ 

```

1. Nous n'extrayons pas que les pires scénarios mais tous les scénarios dépassant un certain seuil sub_{OPT} (variant en fonction de τ), donc nous extrayons des chemins qui se terminent sur des noeuds différents. Nous ne pouvons donc pas normaliser par 2Γ .

4. Algorithme de Sélection MaxMin

Une fois la distance bien définie, nous pouvons implémenter le critère de robustesse **MaxMin** avec pour idée de maximiser la distance minimale. Nous fixons N le nombre de scénarios à renvoyer au Maître. Voici les différentes étapes :

- On insère d'office le pire scénario absolu P_{opt} . Il devient notre référence ;
- Boucle **Min** (calcul du plus proche voisin) : Tant que N n'est pas atteint, on analyse tous les chemins candidats P_{cand} ;
Logique : Si un candidat a une distance min_dist très faible, cela signifie qu'il est très proche d'un autre scénario **déjà sélectionné** ;
- Boucle **Max** : On choisit le candidat dont le plus proche voisin est le plus loin possible.

Algorithm 3 Orthogonality Heuristic (HOG) - Max-Min Selection

Require: Optimal paths P_{opt} , Candidate paths P_{cand} , Number of paths to select N

Ensure: Set of selected orthogonal paths P_{sel}

```

1:  $P_{sel} \leftarrow \emptyset$ 
2: if  $P_{opt} \neq \emptyset$  then                                ▷ Locking in the absolute worst-case scenario for the Master.
3:    $P_{sel} \leftarrow P_{sel} \cup \{P_{opt}[0]\}$ 
4: end if
5: while  $|P_{sel}| < N$  and  $P_{cand} \neq \emptyset$  do
6:    $best\_max\_min \leftarrow -1$ 
7:    $best\_candidate \leftarrow \text{null}$ 
8:   for each  $p_c \in P_{cand}$  do                                ▷ Minimization loop
9:      $min\_dist \leftarrow \infty$ 
10:    for each  $p_s \in P_{sel}$  do                                ▷ For each scenario already selected
11:       $dist \leftarrow \text{BRAY\_CURTIS\_DISTANCE}(p_c, p_s)$       ▷ Dissimilarity calculation
12:      if  $dist < min\_dist$  then
13:         $min\_dist \leftarrow dist$ 
14:      end if
15:    end for
16:    if  $min\_dist > best\_max\_min$  then                                ▷ Maximization loop
17:       $best\_max\_min \leftarrow min\_dist$ 
18:       $best\_candidate \leftarrow p_c$ 
19:    end if
20:  end for
21:   $P_{sel} \leftarrow P_{sel} \cup \{best\_candidate\}$ 
22:   $P_{cand} \leftarrow P_{cand} \setminus \{best\_candidate\}$ 
23: end while
24: return  $P_{sel}$ 

```

Références

- [1] C. RICOTTA et J. PODANI. « On some properties of the Bray-Curtis dissimilarity and their ecological meaning ». In : *Ecological Complexity* 31 (2017), p. 201-205. ISSN : 1476-945X. DOI : <https://doi.org/10.1016/j.ecocom.2017.07.003>. URL : <https://www.sciencedirect.com/science/article/pii/S1476945X17300582>.
- [2] Carlo RICOTTA et Sandrine PAVOINE. « A new parametric measure of functional dissimilarity : Bridging the gap between the Bray-Curtis dissimilarity and the Euclidean distance ». In : *Ecological Modelling* 466 (2022), p. 109880. ISSN : 0304-3800. DOI : <https://doi.org/10.1016/j.ecolmodel.2022.109880>. URL : <https://www.sciencedirect.com/science/article/pii/S0304380022000084>.